

JUnit Testing Revision Notes

Plain-text focused revision material for RAG pipeline testing

1. What is Unit Testing?

Unit testing is the process of testing a small and isolated part of an application, usually a method or class, to verify that it behaves as expected.

A unit test should normally be fast, deterministic, repeatable, isolated, and focused on one behavior.

The main purpose of unit testing is to detect defects early and give developers confidence when changing existing code.

A unit test should not normally depend on a real database, external API, message broker, file system, or network service. Those dependencies can be replaced with mocks or other test doubles when isolation is required.

2. JUnit 5

JUnit 5 is a modern testing framework for Java applications.

JUnit 5 consists of three major parts.

JUnit Platform is the foundation used to launch tests and integrate JUnit with IDEs and build tools.

JUnit Jupiter provides the JUnit 5 programming model and annotations such as `@Test`, `@BeforeEach`, `@AfterEach`, and `@ParameterizedTest`.

JUnit Vintage provides support for running older JUnit 3 and JUnit 4 tests on the JUnit 5 platform.

3. Basic JUnit Test

A basic JUnit 5 test uses the `@Test` annotation.

Example:

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;
```

```
class CalculatorTest {
    @Test
    void shouldAddTwoNumbers() {
        Calculator calculator = new Calculator();
        int result = calculator.add(2, 3);
        assertEquals(5, result);
    }
}
```

The `@Test` annotation tells JUnit that the method is a test method.

The assertion verifies whether the actual result matches the expected result.

4. Important JUnit Assertions

`assertEquals(expected, actual)` verifies that two values are equal.

`assertNotEquals(expected, actual)` verifies that two values are different.

`assertTrue(condition)` verifies that a condition is true.

`assertFalse(condition)` verifies that a condition is false.

`assertNull(value)` verifies that a value is null.

`assertNotNull(value)` verifies that a value is not null.

assertSame(expected, actual) verifies that two variables refer to the same object.

assertNotSame(expected, actual) verifies that two variables refer to different object references.

assertThrows(Exception.class, executable) verifies that a specific exception is thrown.

Example:

```
assertThrows(UserNotFoundException.class, () -> service.getUser(99L));
```

assertAll() allows multiple assertions to be executed together.

Example:

```
assertAll(  
    () -> assertEquals(1L, user.getId()),  
    () -> assertEquals("Rajesh", user.getName()),  
    () -> assertEquals("user@example.com", user.getEmail())  
);
```

5. JUnit Test Lifecycle

JUnit provides lifecycle annotations for setup and cleanup.

@BeforeAll runs once before all tests in a test class. By default, the method must be static.

@BeforeEach runs before every test method. It is commonly used to initialize objects needed by each test.

@AfterEach runs after every test method and can be used for cleanup.

@AfterAll runs once after all tests. By default, the method must be static.

Example:

```
@BeforeEach  
void setUp() {  
    service = new UserService();  
}  
  
@AfterEach  
void tearDown() {  
    service = null;  
}
```

A good test should avoid unnecessary shared state because shared mutable state can make tests dependent on each other.

6. Arrange, Act, Assert Pattern

The Arrange, Act, Assert pattern is a common structure for unit tests.

Arrange means preparing objects, inputs, and mock behavior.

Act means executing the method being tested.

Assert means verifying the expected result or behavior.

Example:

```
@Test  
void shouldReturnUserWhenIdExists() {  
    // Arrange  
    when(repository.findById(1L))  
        .thenReturn(Optional.of(user));
```

// Act

```
User result = service.getUser(1L);
```

// Assert

```
assertEquals(user, result);
```

```
}
```

Keeping these three stages clear makes tests easier to read and maintain.

7. Test Naming

Test names should describe the behavior being tested.

Examples include:

```
shouldReturnUserWhenIdExists()
```

```
shouldThrowExceptionWhenUserDoesNotExist()
```

```
shouldRejectInvalidEmail()
```

```
shouldCreateBookingWhenRequestIsValid()
```

```
shouldNotCreateBookingWhenSalonIsClosed()
```

Meaningful test names make failures easier to understand without opening the implementation.

8. Mockito

Mockito is a mocking framework commonly used with JUnit to isolate the class under test from its dependencies.

For example, a `UserService` may depend on `UserRepository`. Instead of connecting to a real database during a unit test, `UserRepository` can be mocked.

Example:

```
@ExtendWith(MockitoExtension.class)
class UserServiceTest {
    @Mock
    private UserRepository repository;

    @InjectMocks
    private UserService service;
}

@Mock creates a mock dependency.

@InjectMocks creates the class under test and injects the available mocks into it.

@ExtendWith(MockitoExtension.class) enables Mockito support in JUnit 5.
```

9. Mockito when and thenReturn

`when(...).thenReturn(...)` defines what a mocked method should return.

Example:

```
when(repository.findById(1L))
    .thenReturn(Optional.of(user));
```

When `service.getUser(1L)` calls `repository.findById(1L)`, the mock returns the configured `Optional` containing the user.

Another example:

```
when(repository.findById(99L))
    .thenReturn(Optional.empty());
```

This allows the test to simulate a missing user without accessing a real database.

10. Mockito thenThrow

thenThrow is used when a mocked dependency should throw an exception.

Example:

```
when(repository.save(any(User.class)))  
.thenThrow(new RuntimeException("Database error"));
```

This allows you to test how the service handles dependency failures.

11. Mockito verify

verify() checks whether a mocked method was called.

Example:

```
verify(repository).save(user);
```

You can verify an exact number of calls:

```
verify(repository, times(1)).save(user);
```

You can verify that an interaction never happened:

```
verify(repository, never()).deleteById(anyLong());
```

verifyNoMoreInteractions(repository) can be used when you intentionally want to ensure that no additional interactions occurred.

Interaction verification should be used when the interaction is relevant to the behavior being tested. Avoid verifying every internal implementation detail.

12. Mockito Argument Matchers

Mockito provides argument matchers.

any() matches any object.

anyLong() matches any long value.

anyString() matches any string.

any(User.class) matches any User object.

eq(value) matches an exact value.

Example:

```
when(repository.findById(anyLong()))  
.thenReturn(Optional.of(user));
```

Example:

```
verify(repository).findById(eq(1L));
```

When using Mockito matchers in a method call, avoid mixing matchers with raw arguments incorrectly.

13. Mock, Spy, and Real Object

A mock is a test double whose behavior is controlled by Mockito. It does not normally execute the real implementation.

A spy wraps a real object. Real methods can execute unless they are stubbed.

A real object is the actual production implementation.

Mocks are useful for isolating dependencies such as repositories, external APIs, email clients, payment clients, Kafka producers, and vector search services.

Spies should be used carefully because they can make tests more coupled to implementation details.

14. Testing Exceptions

JUnit 5 provides `assertThrows()` for testing exceptions.

Example:

```
@Test
void shouldThrowWhenUserDoesNotExist() {
    when(repository.findById(99L))
        .thenReturn(Optional.empty());
```

```
assertThrows(
    UserNotFoundException.class,
    () -> service.getUser(99L)
);
}
```

If the exception message is part of the application's contract, the exception can be stored and its message can also be verified.

Example:

```
UserNotFoundException exception =
assertThrows(
    UserNotFoundException.class,
    () -> service.getUser(99L)
);

assertEquals("User not found", exception.getMessage());
```

15. Parameterized Tests

Parameterized tests allow the same test logic to run with multiple input values.

Example:

```
@ParameterizedTest
@ValueSource(strings = {"abc", "hello", "spring"})
void shouldAcceptNonEmptyStrings(String value) {
    assertFalse(value.isBlank());
}
```

Common parameterized test sources include `@ValueSource`, `@CsvSource`, `@MethodSource`, `@NullSource`, and `@EmptySource`.

Parameterized tests reduce duplicated test code and are useful when the same behavior should be checked for many inputs.

16. Testing Spring Boot Services

A service-layer unit test should normally test business logic without loading the entire Spring application.

For example, if `TutorService` depends on `MaterialRepository` and `VectorStoreService`, those dependencies can be mocked.

Test successful behavior, validation errors, missing data, authorization or ownership rules, exceptions, and different dependency responses.

Example:

```
@Mock
private MaterialRepository materialRepository;
```

```
@Mock
private VectorStoreService vectorStoreService;

@InjectMocks
private TutorService tutorService;
```

This approach keeps service tests fast and focused.

17. @SpringBootTest

@SpringBootTest loads a broad Spring application context.

It is useful when you want to test how multiple Spring components work together.

Example:

```
@SpringBootTest
class ApplicationIntegrationTest {
    @Test
    void contextLoads() {
    }
}
```

Because @SpringBootTest can start many parts of the application, it is generally heavier and slower than a pure unit test.

Use it when the behavior being tested actually requires the Spring application context.

18. @WebMvcTest

@WebMvcTest is used for focused Spring MVC controller tests.

It loads a web-related test context instead of the complete application.

MockMvc can be used to send simulated HTTP requests to controller endpoints.

Example:

```
@WebMvcTest(UserController.class)
class UserControllerTest {
    @Autowired
    MockMvc mockMvc;

    @MockBean
    UserService userService;

    @Test
    void shouldGetUser() throws Exception {
        when(userService.getUser(1L))
            .thenReturn(user);

        mockMvc.perform(get("/api/users/1"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.id").value(1));
    }
}
```

Depending on the Spring Boot version, newer projects may use the current Mockito-based Spring test annotation provided by that version instead of @MockBean.

19. MockMvc

MockMvc allows controller endpoints to be tested without starting a real HTTP server.

It can test HTTP methods, request parameters, request bodies, response status codes, response JSON, headers, and validation behavior.

Common examples:

```
mockMvc.perform(get("/api/users/1"))
```

```
mockMvc.perform(post("/api/users"))
```

```
.andExpect(status().isOk())  
.andExpect(status().isBadRequest())  
.andExpect(jsonPath("$.name").value("Rajesh"))
```

MockMvc is especially useful for testing REST controllers.

20. @DataJpaTest

@DataJpaTest is designed for testing JPA and repository behavior.

It loads a focused persistence-related Spring test context.

It can be used to test repository queries, entity mappings, relationships, and database-related behavior.

Repository tests are different from pure service unit tests because they may interact with a test database or configured persistence layer.

21. Unit Test vs Integration Test

A unit test focuses on one unit and usually mocks external dependencies.

An integration test checks how multiple components work together.

For example, a UserServiceTest may mock UserRepository and test only UserService business logic.

An integration test might load Spring, connect to a test database, and verify that the service and repository work together.

Both types are useful, but they answer different questions.

22. Testing REST APIs

When testing a REST API, important cases include successful requests, invalid input, missing resources, unauthorized requests, forbidden access, duplicate resources, validation errors, and unexpected dependency failures.

For each endpoint, verify the HTTP status and important response fields.

For request validation, test both valid and invalid payloads.

For ownership-based APIs, test that a user can access their own resource and that access to another user's resource is rejected according to the application's authorization rules.

23. Testing Boundary Cases

Good unit tests include normal cases and boundary cases.

Examples include empty strings, null input, zero, negative numbers, maximum allowed values, duplicate values, missing database records, and invalid IDs.

For pagination, useful boundary cases include page zero, the first page, the last page, an empty page, and an invalid page size.

Boundary testing is useful because many defects occur around limits rather than normal inputs.

24. Common Unit Testing Mistakes

A common mistake is testing implementation details instead of observable behavior.

Another mistake is using a real database for every unit test, which can make tests slow and fragile.

Another mistake is forgetting to stub a mocked dependency, causing unexpected null or default values.

Tests should not depend on execution order.

Tests should not share mutable state unnecessarily.

Avoid one huge test that checks many unrelated behaviors.

Do not create tests that pass only because the test duplicates the implementation logic.

Use meaningful test names and keep each test focused.

25. Maven Commands for JUnit

```
mvn test runs the test suite.  
mvn clean test cleans the project and runs tests.  
mvn -Dtest=UserServiceTest test runs a specific test class.  
mvn -Dtest=UserServiceTest#shouldReturnUser test runs a specific test method.  
mvn clean compile compiles the project without running the tests.
```

For a Spring Boot Maven project, tests are normally located under `src/test/java`.

26. JUnit Interview Questions

Question: What is unit testing?

Answer: Unit testing verifies a small unit of application behavior in isolation.

Question: What is the difference between JUnit and Mockito?

Answer: JUnit provides the testing framework, test lifecycle, and assertions. Mockito provides mocks and tools for controlling and verifying dependencies.

Question: What is `@Mock`?

Answer: `@Mock` creates a Mockito mock object.

Question: What is `@InjectMocks`?

Answer: `@InjectMocks` creates the class under test and injects available Mockito mocks into it.

Question: What is `@BeforeEach`?

Answer: It runs before every test method.

Question: How do you test an exception in JUnit 5?

Answer: Use `assertThrows()`.

Question: What is `@SpringBootTest`?

Answer: It loads a broad Spring application context and is commonly used for integration-style testing.

Question: What is `@WebMvcTest`?

Answer: It loads a focused Spring MVC test context for controller testing.

Question: What is `MockMvc`?

Answer: `MockMvc` is a Spring testing utility used to test MVC endpoints without starting a real HTTP server.

Question: Why use Mockito?

Answer: Mockito allows dependencies to be replaced with controllable test doubles so the class under test can be isolated.

Question: What is the AAA pattern?

Answer: Arrange, Act, Assert. Arrange prepares the test, Act executes the behavior, and Assert verifies the result.

27. JUnit Testing Checklist

Test the successful path.

Test invalid input.

Test missing data.

Test important boundary values.

Test expected exceptions.

Test authorization or ownership rules where applicable.

Mock external dependencies for unit tests.

Verify important interactions when those interactions are part of the behavior.

Keep tests independent.

Use descriptive test names.

Run the complete test suite before committing changes.

28. JUnit Testing for an AI Study Companion

For an AI Study Companion application, services such as TutorService, AssessmentService, MasteryService, and RecommendationService can be unit tested independently.

For TutorService, mock the repository, vector search service, AI client, and other external dependencies.

Test that a valid question produces the expected response.

Test behavior when no relevant study material is found.

Test behavior when a project or material does not exist.

Test authorization or ownership checks.

Test repository failures and AI-service failures where the application has defined handling for them.

For AssessmentService, test quiz creation, answer validation, scoring, invalid quiz IDs, missing questions, and other business rules.

For MasteryService, test mastery updates and boundary cases.

For RecommendationService, test different mastery levels and the resulting recommendation logic.

The goal is to verify application behavior without making real external AI calls during unit tests.

29. Quick Revision Summary

JUnit 5 provides the test framework and assertions.

```
@Test marks a test method.  
@BeforeEach runs before every test.  
@AfterEach runs after every test.  
@BeforeAll runs once before all tests.  
@AfterAll runs once after all tests.
```

assertEquals checks equality.

assertTrue and assertFalse check boolean conditions.

assertNull and assertNotNull check nullability.

assertThrows checks exceptions.

Mockito creates and controls mocks.

```
@Mock creates a mock dependency.  
@InjectMocks creates the class under test and injects mocks.
```

when().thenReturn() defines mock behavior.

verify() checks interactions.

```
@ParameterizedTest runs one test with multiple inputs.  
@SpringBootTest loads a broad Spring context.  
@WebMvcTest focuses on controller testing.
```

MockMvc tests MVC endpoints without starting a real HTTP server.

```
@DataJpaTest focuses on JPA and repository testing.
```

Unit tests isolate one unit. Integration tests verify multiple components working together.